

Stacks

LEARNING OBJECTIVES

After studying this chapter, readers will be able to:

- Understand stack and its operations
- Implement stack using list
- Implement stack using queue module
- Implement stack using dictionary
- Implement stack using class
- Implement stack using queue module
- Apply stack to match delimiter
- Apply stack for infix–postfix conversion
- Apply stack for evaluation of the postfix expression
- Apply stack for recursion

We know that the linear data structures are divided into arrays, stacks and queues. In this chapter, we will focus on the stack as a linear data structure. Stack data structures can be implemented using arrays. Simple examples of a stack are a stack of plates in a restaurant and a stack of books in a library.

5.1 STACKS

The insert and delete operations are done at the same end of a stack, which is a special case of an ordered list. Stacks are used as a data structure in programming languages to achieve recursion, to transform an infix expression to a postfix expression and so on.

A stack is a restricted ordered list in which only one end, termed the *top* of the stack, allows insertions and deletions. The stack abstract data type has the data member *top*, which points to the topmost element in the stack. There are two basic operations: *push()* adds an element to the top of the stack and *pop()* removes an element from the top of the stack. The element

that is inserted last is deleted first. So a stack is also known as a *last-in-first-out (LIFO)* data structure.

If we describe the stack as $S = \{a_1, a_2, \dots, a_n\}$, then we consider a_1 to be at the bottom of the stack and a_n as at the top of the stack and a_{i+1} as on the top of a_i .

A stack with four elements can be pictorially represented as in Figure 5.1.

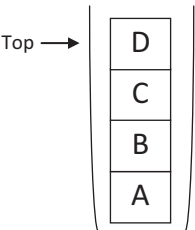


Figure 5.1 Stack

5.2 STACK OPERATIONS

The basic operations on a stack are the following:

Description	Operation name
Push an element onto the stack	push(element)
Pop an element from a stack	pop()
Get the top element of the stack	peek()
Display the elements in the stack	display()

5.2.1 push()

The *push(S)* operation inserts an element on the top of the stack S (Figure 5.2). It follows these steps:

1. Let *top* be the index of the topmost element.
2. Check if the stack is full (if it is a restricted stack).
3. If stack is not full.
 - a. Increment *top* by one.
 - b. Top of the stack = $S[\text{top}]$ (S is the list we use to create the stack).
 - c. Return stack.
4. Else report failure (*stack overflow*).

The maximum size of a Python list on a 32 bit system is around 536,870,912 elements. Since we are using a list to implement a stack, the condition *isFull()* is not required. If we implement a restricted list (a list with predefined size),

then *isFull()* is required to be implemented. We increment the 'top' by one and insert $S[\text{top}]$ onto the stack.

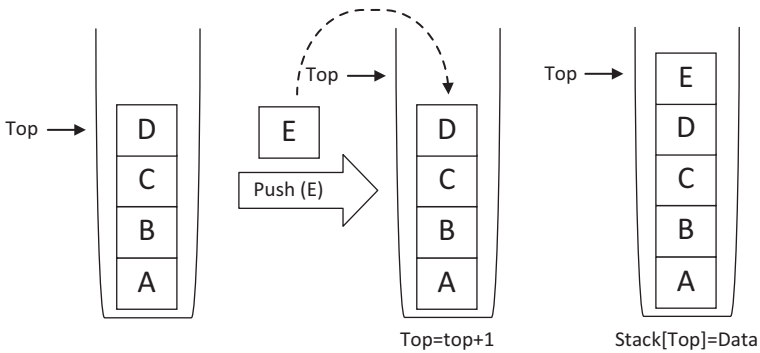


Figure 5.2 Stack operation push

5.2.2 pop()

We execute the following steps to perform the pop operation (also see Figure 5.3):

1. Let x be the variable to store the popped element.
2. If the stack is not empty.
 - a. Set $x = A[\text{top}]$.
 - b. Decrement top by one.
 - c. Return stack.
3. Else report failure (*stack underflow*).

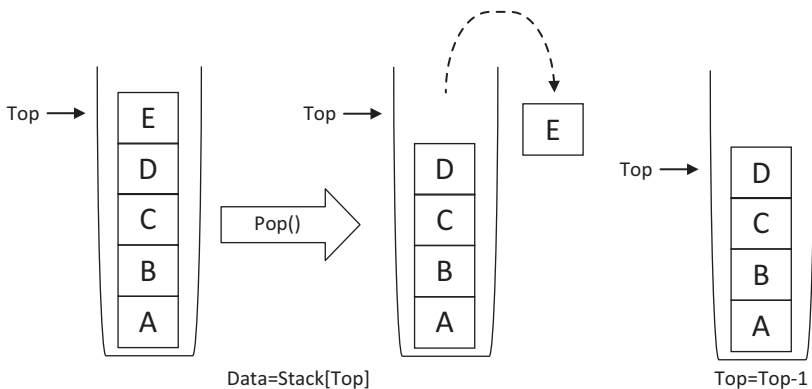


Figure 5.3 Stack operation pop

5.2.3 peek()

peek() is similar to *pop()* except that this function removes the element from the top of the stack without decrementing the top by one (Figure 5.4).

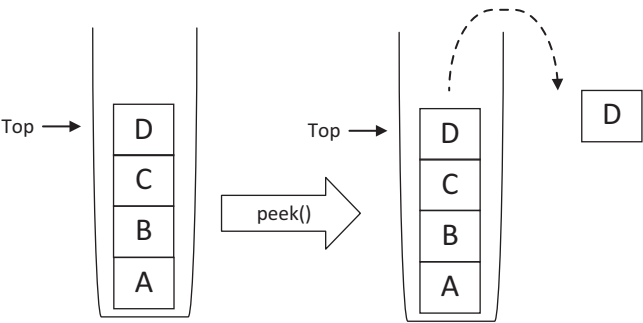


Figure 5.4 Stack operation peek

An error condition (*stack underflow*) is reported if we try to remove an element from an empty stack.

5.3 STACK (ADT)

Let us formally describe the structure of the stack together with a set of axioms that we expect for the stacks.

The abstract data type stack can be described by the following methods and axioms.

Methods: See Table 5.1.

Table 5.1 Stack Methods

Method	Description
Create()	Creates an empty stack, say, S.
push(i,S)	Pushes i onto the stack: returns the resultant stack.
pop(S)	Pops the top element from the stack: returns the resultant stack.
peek()	Displays the top element: stack remains the same.
isEmpty(S)	Returns true if the stack is empty (top=-1 for an empty stack).
isFull(S)	Returns true if the stack is Full (Applicable for fixed size stack).

Axioms: See Table 5.2.

Table 5.2 Stack axioms with return values

Axiom	Return value
<code>isEmpty(create())</code>	True
<code>isEmpty(push(e,S))</code>	False
<code>pop(create())</code>	Error
<code>peek(create())</code>	Error
<code>peek(push(e,S))</code>	Error

The first axiom says that when we create a stack, it is empty, hence the *isEmpty()* boolean function returns *true*.

The second axiom is easy to understand since when we push an element onto a stack, it is not empty, hence the *isEmpty()* function returns *false*.

We cannot *pop()* an element from an empty stack. Thus *pop(create())* gives an *error* since the *create()* function creates an empty stack and so is the case with *peek(create())*.

peek(push(e,S)) returns *e* since after the push operation the topmost element is *e*. Also see Table 5.3.

Table 5.3 Stack methods with return

Function	Return
<code>create (S)</code>	Stack
<code>push(i,s)</code>	Stack
<code>pop(S)</code>	Element
<code>getTop(S)</code>	Element
<code>isEmpty(s)</code>	Boolean
<code>isEmpty(create(S))</code>	True
<code>isEmpty(push(E,S))</code>	False
<code>pop(create(S))</code>	Error
<code>peek(create(S))</code>	Error
<code>peek(push(E,S))</code>	Error

5.4 IMPLEMENTATION

There are various ways to implement a stack. It can be implemented using a list, dictionary or a class in Python.

5.4.1 Implementation using a list

The elements of the stack are the elements of a list. As a list is not a fixed-size data structure, stack is also not of fixed size while implementing. But we can restrict the size of the list, hence the size of stack can be made fixed.

Now let us implement a stack using list without restriction of the upper limit or the size of the stack.

Python program 1

```
#Create an empty stack
def create():
    S=[]
    top = -1
    print("Empty Stack created")
    return S,top
```

The create() method creates an empty stack with the top pointing to -1. Here S = [] indicates an empty list.

Program continues...

```
#Checks if the stack is empty
def isEmpty(top):
    if (top <= -1):
        return 1
    else:
        return 0
```

The isEmpty() method checks if the stack is empty. It checks if the top is pointing to -1. It returns 1 to indicate the stack is empty; otherwise returns 0 to indicate the stack is not empty (at least one element is there in the stack).

Program continues...

```
#Insert data into stack
def push(S,top):
    no = int(input(("Enter a number to push onto the
        stack ")))
    top += 1
    S.append(no)
    return S,top
```

The push() method takes two parameters, the stack and the top. Every time an element is pushed onto the stack, the top is incremented by 1 and the element is appended to the list. As a list is a heterogeneous data structure, we force the user to enter an integer only.

Program continues...

```
#Delete from stack
def pop(S,top):
    if (isEmpty(top) == 1):
        print("Underflow: There is no element in the Stack")
    else:
        print("The element to pop is ",S[top])
        top -= 1
        return S,top
```

The `pop()` method takes two parameters, the stack and the top. Every time an element is popped from the stack, it checks if the stack is not empty. If the stack is empty, it shows an appropriate message, and if the stack is not empty, then the top of the stack is printed and the top is decremented by 1. At last the stack and the top is returned.

Program continues...

```
#Returns the top of the stack
def peek(S,top):
    if (isEmpty(top) == 1):
        print("Stack is Empty")
    else:
        print("The element at top is ",S[top])
```

The `peek()` method takes two parameters, the stack and the top. It just prints the element that is on the top of the stack.

Program continues...

```
#Displays the stack element
def display(S,top):
    print("Top is at index -> ",top)
    if (isEmpty(top) == 1) :
        print("\nStack is Empty")
    else:
        print("The elements in the Stack is")
        for i in range(top,-1,-1):
            print(i, "->",S[i])
```

The `display()` method takes two parameters, the stack and the top. This function prints the elements in the stack in the reverse order (the element inserted last is printed first).

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Push an element ")
    print("2. Pop from stack ")
    print("3. Peek the stack")
    print("4. Display the stack")
    print("5. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

The menu() method shows a menu to perform the operations defined for a stack. It returns a choice of operation to perform.

Program continues...

```
#Main Program
if __name__ == "__main__":
    St,top=create()
    i=1
    while (i > 0 and i < 5 ):
        i = menu()
        if i == 1:
            St,top = push(St,top)
            display(St,top)
        elif i == 2:
            St,top = pop(St,top)
            if (type(top) != str):
                display(St,top)
        elif i == 3:
            peek(St,top)
        elif i == 4:
            display(St,top)
        else:
            print("Exit")
```

Output

Empty Stack created

```
~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 99
Top is at index -> 0
The elements in the Stack is
0 -> 99
```

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 1

Enter a number to push onto the stack 88

Top is at index -> 1

The elements in the Stack is

1 -> 88

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 1

Enter a number to push onto the stack 77

Top is at index -> 2

The elements in the Stack is

2 -> 77

1 -> 88

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 3

The element at top is 77

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 77

Top is at index -> 1

The elements in the Stack is

1 -> 88

0 -> 99

```
~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 2
The element to pop is 88
Top is at index -> 0
The elements in the Stack is
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 2
The element to pop is 99
Top is at index -> -1

Stack is Empty

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 2
Underflow: There is no element in the Stack
Top is at index -> -1

Stack is Empty

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 6
Exit
```

Now let us implement a stack using list with a restriction of upper limit or the size of the stack. There is no change in the create(), pop(), peek(), isEmpty() and menu() functions and some changes in the push() and display() functions. A new function isFull() is also introduced. We are discussing only those functions that have some changes from Python program 1.

Python program 2

```
def isFull(top,size):
    if (top >= size-1):
        return 1
    else:
        return 0
```

The `isFull()` method checks if the stack is full. It checks if the top is pointing to maximum size -1. It returns 1 to indicate the stack is full; otherwise returns 0 to indicate the stack is not full.

Program continues...

```
def push(S,top,size):
    if (isFull(top,size) == 1):
        print("Overflow: No more place for the Stack")
    else:
        no = int(input(("Enter a number to push onto
            the stack ")))
        top += 1
        S.append(no)
    return S,top
```

The `push()` method takes two parameters, the stack and the top. Every time an element is pushed onto the stack, it checks if the stack is full. If the stack is full, it shows an appropriate message; otherwise the top is incremented by one and the element is appended to the list.

Program Continues...

```
def display(S,top,size):
    print("Top is at index -> ",top)
    if (isEmpty(top) == 1) :
        print("\nStack is Empty")

    elif(isFull(top,size) == 1):
        print("\nStack is Full")
    else:
        print("The elements in the Stack is")
        for i in range(top,-1,-1):
            print(i, "->",S[i])
```

The `display()` method takes two parameters, the stack and the top. This function prints the elements in the stack in the reverse order (the element inserted last is printed first) if the stack is neither full nor empty.

Output

```
Enter the size of the Stack 3
Empty Stack created

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 99
Top is at index -> 0
The elements in the Stack is
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 88
Top is at index -> 1
The elements in the Stack is
1 -> 88
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 77
Top is at index -> 2

Stack is Full

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Overflow: No more place for the Stack
Top is at index -> 2
```

Stack is Full

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 3

The element at top is 77

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 77

Top is at index -> 1

The elements in the Stack is

1 -> 88

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 88

Top is at index -> 0

The elements in the Stack is

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 99

Top is at index -> -1

Stack is Empty

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack

```
4. Display the stack
5. Exit
Enter a valid menu item ... 5
Exit
```

### 5.4.2 Implementation of stack using a dictionary

Now let us implement a stack using dictionary with a restriction of upper limit or the size of the stack. We already know that a dictionary is a key-value pair. We are implementing the stack with a size restriction.

#### Python program 3

```
def create():
    S={}
    top = -1
    print("Empty Stack created")
    return S,top
```

In create() method, we have created an empty dictionary S={} and assigned top to -1.

#### Program continues...

```
def isFull(top,size):
    if (top >= size-1):
        return 1
    else:
        return 0
```

Just like the previous program, the isFull() function checks if the stack is full. When the location of the top is greater than or equal to maximum size -1, it returns 1 indicating the stack is full; otherwise it returns 0 indicating the stack is not full.

#### Program continues...

```
def isEmpty(top):
    if (top <= -1):
        return 1
    else:
        return 0
```

The isEmpty() function checks if the stack is empty. When the location of the top is less than or equal to -1, it returns 1 indicating the stack is empty; otherwise it returns 0 indicating the stack is not empty.

**Program continues...**

```
def push(S,top,size):
    if (isFull(top,size) == 1 ):
        print("Overflow: No more place in the Stack")

    else:
        no = int(input(("Enter a number to push onto the stack ")))
        top += 1
        S[top] = no
    return S,top
```

The push()function takes three parameters: the stack S, top and the size of the stack. If the stack is full, it prints an appropriate message; otherwise increment the top by one and insert the element on the top of the stack. It returns the stack and the top of the stack.

**Program continues...**

```
def pop(S,top):
    if (isEmpty(top) == 1):
        print("Underflow: There is no element in the Stack")
    else:
        print("The element to pop is ",S[top])
        del S[top]
        top -= 1
    return S,top
```

The pop()function takes two parameters: the stack S and top of the stack. If the stack is empty, it prints an appropriate message; otherwise, decrement the top by 1 and return the stack and the top.

**Program continues...**

```
def peek(S,top):
    if (isEmpty(top) == 1):
        print("Stack is Empty")
    else:
        print("The element at top is ",S[top])
```

The peek() method takes two parameters, the stack and the top. It just prints the element that is on the top of the stack.

**Program continues...**

```
def display(S,top,size):
    print("Top is at index -> ",top)
    if (isEmpty(top) == 1) :
        print("\nStack is Empty")

    elif(isFull(top,size) == 1):
        print("The elements in the Stack is")
        for i in reversed(list(S.keys())):
            print(i, "->",S[i])
        print("\nStack is Full")

    else:
        print("The elements in the Stack is")
        for i in reversed(list(S.keys())):
            print(i, "->",S[i])
```

The `display()` method takes two parameters: the stack and the top. This function prints the elements in the stack in the reverse order (the element inserted last is printed first) if the stack is neither full nor empty.

**Program continues...**

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Push an element ")
    print("2. Pop from stack ")
    print("3. Peek the stack")
    print("4. Display the stack")
    print("5. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

**Program continues...**

```
if __name__ == "__main__":
    size = int(input("Enter the size of the Stack "))
    St,top=create()
    i=1
    while (i > 0 and i < 5 ):
        i = menu()
        if i == 1:
            St,top = push(St,top,size)
            display(St,top,size)
        elif i == 2:
            St,top = pop(St,top)
            display(St,top,size)
        elif i == 3:
            peek(St,top)
```

```
elif i == 4:
    display(St,top,size)
else:
    print("Exit")
```

The output of the program is the same as the previous program.

### 5.4.3 Implementation of stack using a class

Now let us implement a stack using class with a restriction of upper limit or the size of the stack.

#### Python program 4

```
class Stack:
    def __init__(self,size):
        self.S = list()
        self.top = -1
        self.maxSize = size
        print("\nEmpty Stack Created")
```

Class 'Stack' is defined with three variables: a list, 'top' initialized to -1 and maximum size of the stack. All the methods in the class are public.

#### Program continues...

```
def isFull(self):
    if (self.top >= self.maxSize-1):
        return 1
    else:
        return 0
```

The `isFull()` method is a Boolean method that returns 1 (true) if the stack full condition is satisfied `self.top >= self.maxSize-1` and returns 0 (false) otherwise.

#### Program continues...

```
def isEmpty(self):
    if (self.top <= -1):
        return 1
    else:
        return 0
```

The `isEmpty()` method is a Boolean method that returns 1 (true) if the empty condition is satisfied `self.top <= -1` and returns 0 (false) otherwise.

**Program continues...**

```
def push(self):  
    if (self.isFull() == 1):  
        print("Overflow: No more place for the Stack")  
    else:  
        no = int(input(("Enter a number to push onto the stack ")))  
        self.top += 1  
        self.S.append(no)
```

The push() function first checks if the stack is full. If the stack is full, it prints an appropriate message; otherwise, increment the top by one and insert the element on the top of the stack.

The initial stack is shown in Figure 5.5.

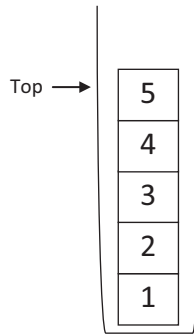


Figure 5.5 Stack push method

After executing push() method, the stack status is as shown in Figure 5.6.

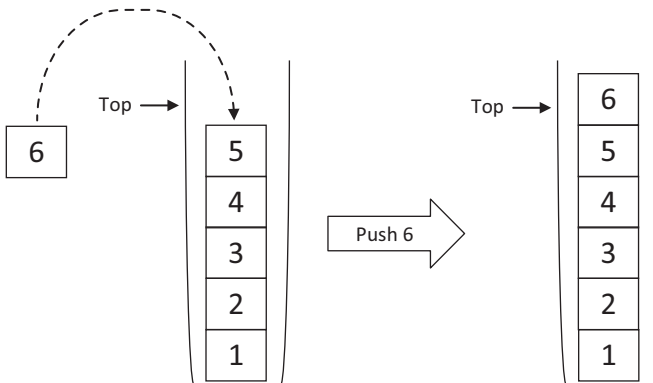


Figure 5.6 Stack push(6) method

**Program continues...**

```
def pop(self):  
    if (self.isEmpty()==1):  
        print("Underflow: There is no element in the Stack")  
    else:  
        print("The element to pop is ",self.S[self.top])  
        self.top -= 1
```

The `pop()` function first checks if the stack is empty. If the stack is empty, it prints an appropriate message; otherwise it prints the element at the top of the stack and then decrements the top by one.

The initial stack is shown in Figure 5.7.

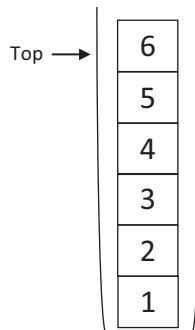


Figure 5.7 Sample stack

After the execution of `pop()`, the topmost element pops-out and leaves the stack as shown in Figure 5.8.

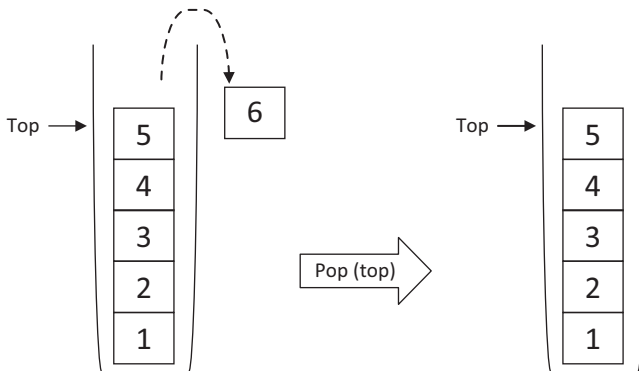


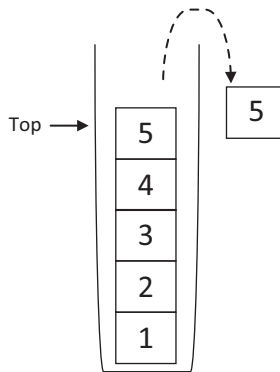
Figure 5.8 Sample stack pop operation

**Program continues...**

```
def peek(self):  
    if(self.isEmpty()==1):  
        print("Stack is Empty")  
    else:  
        print("The element at top is ",self.S[self.top])
```

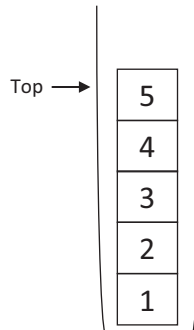
The peek() function first checks if the stack is empty. If the stack is empty, it prints an appropriate message; otherwise it prints the element at the top of the stack.

The initial stack is shown in Figure 5.9.



*Figure 5.9* Stack before execution of peek operation

The execution of peek() leaves the stack as shown in Figure 5.10.



*Figure 5.10* Stack after execution of peek operation

**Program continues...**

```
def display(self):
    print("Top is at index -> ",self.top)
    if(self.isEmpty()==1):
        print("\nStack is Empty")

    elif(self.isFull()==1):
        for i in range(self.top,-1,-1):
            print(i, "->",self.S[i])
        print("\nStack is Full")

    else:
        print("The elements in the Stack is")
        for i in range(self.top,-1,-1):
            print(i, "->",self.S[i])
```

**Program continues...**

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Push an element ")
    print("2. Pop from stack ")
    print("3. Peek the stack")
    print("4. Display the stack")
    print("5. Exit ")
    opt = int(input("Enter a valid menu item ... "))
    return opt
```

**Program continues...**

```
if __name__ == "__main__":
    size = int(input("Enter the size of the Stack "))
    St = Stack(size)
    i=1
    while (i > 0 and i < 5 ):
        i = menu()
        if i == 1:
            St.push()
            St.display()
        elif i == 2:
            St.pop()
            St.display()
        elif i == 3:
            St.peak()
        elif i == 4:
            St.display()
        else:
            print("Exit")
```

## Output

```
Enter the size of the Stack 3
Empty Stack created

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 99
Top is at index -> 0
The elements in the Stack is
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 88
Top is at index -> 1
The elements in the Stack is
1 -> 88
0 -> 99

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Enter a number to push onto the stack 77
Top is at index -> 2

Stack is Full

~~~ MENU ~~~
1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit
Enter a valid menu item ... 1
Overflow: No more place for the Stack
Top is at index -> 2
```

Stack is Full

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 3

The element at top is 77

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 77

Top is at index -> 1

The elements in the Stack is

1 -> 88

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 88

Top is at index -> 0

The elements in the Stack is

0 -> 99

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack
4. Display the stack
5. Exit

Enter a valid menu item ... 2

The element to pop is 99

Top is at index -> -1

Stack is Empty

~~~ MENU ~~~

1. Push an element
2. Pop from stack
3. Peek the stack

```
4. Display the stack
5. Exit
Enter a valid menu item ... 5
Exit
```

5.4.4 Implementation of stack using queue module

Python program 5

```
import queue
def Push(s):
    if (not s.full()):
        data = input(("Enter a number to insert into the Stack "))
        s.put(data)
    else:
        print("There is NO SPACE in the Stack")
```

Program continues...

```
def Pop(s):
    if (not s.empty()):
        data=s.get()
        print("The Item Deleted is : ",data)
    else:
        print("There is NO DATA in the Stack")
```

Program continues...

```
def Display(s):
    if(s.full()):
        print("The Elements in the Stack are :")
        for i in s.queue:
            print(i)
        print("\nThe Stack is Full")
    elif(s.empty()):
        print("\nThe Stack is EMPTY")
    else:
        print("The Elements in the Stack are :")
        for i in s.queue:
            print(i)
```

Program continues...

```
def menu():
    print("\n~~~ MENU ~~~ ")
    print("1. Push into Stack ")
    print("2. Pop from Stack ")
```

```
print("3. Display the Stack")
print("4. Exit ")
opt = int(input("Enter a valid menu item ... "))
return opt
```

Program continues...

```
if __name__ == "__main__":
    size = int(input("Enter the size of the Stack "))
    St = queue.LifoQueue(size)
    i=1
    while (i > 0 and i < 4 ):
        i = menu()
        if (i == 1):
            Push(St)
            Display(St)
        elif (i == 2):
            Pop(St)
            Display(St)
        elif (i == 3):
            Display(St)
        else:
            print("Exit")
```

Output

```
Enter the size of the Stack 3

~~~ MENU ~~~
1. Push intoto Stack
2. Pop from Stack
3. Display the Stack
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Stack 99
The Elements in the Stack are :
99

~~~ MENU ~~~
1. Push intoto Stack
2. Pop from Stack
3. Display the Stack
4. Exit
Enter a valid menu item ... 1
Enter a number to insert into the Stack 88
The Elements in the Stack are :
99
88
```

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 1

Enter a number to insert into the Stack 77

The Elements in the Stack are :

99

88

77

The Stack is Full

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 2

The Item Deleted is : 77

The Elements in the Stack are :

99

88

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 2

The Item Deleted is : 88

The Elements in the Stack are :

99

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 2

The Item Deleted is : 99

The Stack is EMPTY

~~~ MENU ~~~

1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit

Enter a valid menu item ... 3

The Stack is EMPTY

```

~~~ MENU ~~~
1. Push into Stack
2. Pop from Stack
3. Display the Stack
4. Exit
Enter a valid menu item ... 4
Exit

```

## 5.5 APPLICATIONS OF STACK

Stacks have various applications and are widely used in programming and data structures. In this section, we will discuss delimiter matching, infix-postfix conversion and evaluation of the postfix expression as examples of stacks.

### 5.5.1 Delimiter matching

Generally, a stack is very useful in situations when data have to be stored and retrieved in the reverse order. Let us consider the simple case where the delimiters in an arithmetic expression are '(', ')', '[', ']', '{', '}'. An arithmetic expression is said to be properly formed if the left delimiters and the corresponding right delimiters match. A Python program compiles correctly if and only if they match. Some examples where the delimiters match are

$$a = b + (c - d) + (e - f)$$

$$p[10] = h[(a + b)] + (j + k) - 1$$

A delimiter mismatch occurs in the following arithmetic expressions:

$$a = (b + c) - (d + e))$$

$$p[10] = h[(a + b) - (c + d)$$

A particular delimiter can be separated from its match by other delimiters. In other words, the first delimiter may be matched with the last delimiter; but this is done after the second delimiter is matched with the next to the last delimiter, etc. The delimiter matching algorithm reads a character from a Python program, then stores it in a stack if it is an opening delimiter. If it is a closing delimiter, it is compared with the delimiter popped off the stack. If they match, the processing continues. Otherwise the processing is discontinued and an error is signaled. The processing of the Python program ends successfully if the end of the program is reached and the stack is empty.

Let us write a simple algorithm to match delimiters in an arithmetic expression instead of the whole Python program.

**Algorithm delimiter matching (arithmetic expression)**

```
1. while (not end of expression)
2. read a char ch from the expression
3. if (ch='(' or '[' or '{')
4. push ch onto the stack
5. continue // go to the beginning of the loop
6. if (ch = ')' or ']' or '}')
7. compare ch with top of stack element
8. if they match pop the element from the stack
9. continue
10. else
11. stop and report failure
12. ignore all other characters
13. end
```

Let us apply this algorithm to match the parentheses in the following expression:

`a[5]+(b-{c*(d-e)+(f-g)})`

See Table 5.4. The stack is empty and there is no more input to be read. Thus, the parentheses in the arithmetic expression match.

A simple program to illustrate use of stack for parentheses matching is given next. The program takes a string of parentheses (an arithmetic expression devoid of operands and operators) and checks for matching parentheses.

**Python program 6**

```
def isEmpty(top):
 if (top <= -1):
 return 1
 else:
 return 0

def push(Stack,i,top):
 top += 1
 S.append(i)
 return S,top

def pop(S,top):
 del S[top]
 return S,top-1

def peek(S,top):
 if (isEmpty(top) == 1):
 print("Stack is Empty: Mismatch")
 return 0
```

Table 5.4 Stack operation for delimiter matching

| Contents of the stack | Non-blank character read | Remaining exp                       |
|-----------------------|--------------------------|-------------------------------------|
| Empty                 | -                        | $a[5] + (b - \{c^*(d-e) + (f-g)\})$ |
| Empty                 | A                        | $[5] + (b - \{c^*(d-e) + (f-g)\})$  |
| Empty                 | [                        | $5] + (b - \{c^*(d-e) + (f-g)\})$   |
| [                     | 5                        | $] + (b - \{c^*(d-e) + (f-g)\})$    |
| [                     | ]                        | $+ (b - \{c^*(d-e) + (f-g)\})$      |
| Empty                 | +                        | $(b - \{c^*(d-e) + (f-g)\})$        |
| Empty                 | (                        | $b - \{c^*(d-e) + (f-g)\})$         |
| (                     | B                        | $- \{c^*(d-e) + (f-g)\})$           |
| (                     | -                        | $\{c^*(d-e) + (f-g)\})$             |
| (                     | {                        | $c^*(d-e) + (f-g)\})$               |
| {{                    | C                        | $* (d-e) + (f-g)\})$                |
| {{                    | *                        | $(d-e) + (f-g)\})$                  |
| {{                    | (                        | $d-e) + (f-g)\})$                   |
| {{{                   | D                        | $-e) + (f-g)\})$                    |
| {{{                   | -                        | $e) + (f-g)\})$                     |
| {{{                   | E                        | $) + (f-g)\})$                      |
| {{{                   | )                        | $+ (f-g)\})$                        |
| {{                    | +                        | $(f-g)\})$                          |
| {{                    | (                        | $f-g)\})$                           |
| {{{                   | F                        | $-g)\})$                            |
| {{{                   | -                        | $g)\})$                             |
| {{{                   | G                        | $)\})$                              |
| {{{                   | )                        | $\})$                               |
| {{                    | }                        | $)$                                 |
| (                     | )                        | -                                   |
| Empty                 | -                        | -                                   |

```

else:
 return S[top]

if __name__ == "__main__":
 Expr=input("Enter an Expression ")
 S=[]
 top = -1

 for i in range(len(Expr)):
 if (Expr[i] == '('):
 S,top = push(S,Expr[i],top)

 elif(Expr[i] == '['):
 S,top = push(S,Expr[i],top)

 elif(Expr[i] == '{'):
 S,top = push(S,Expr[i],top)

```

```
elif(Expr[i] == ')'):
 ele= peek(S,top)
 if (ele == '(' and top != -1):
 S,top = pop(S,top)
 else:
 break

elif(Expr[i] == ']'and top != -1):
 ele= peek(S,top)
 if (ele == '['):
 S,top = pop(S,top)
 else:
 break

elif(Expr[i] == '}'and top != -1):
 ele= peek(S,top)
 if (ele == '{'):
 S,top = pop(S,top)
 else:
 break

else:
 continue

if (Expr != '' and top == -1 and ele != 0):
 print("The Expression is well Parenthesized..")
else:
 print("Parenthesis Does not Matched..")
```

### Output 1

```
Enter an Expression a[5]+(b-{c*(d-e)+(f-g)})
The Expression is well Parenthesized..
```

### Output 2

```
Enter an Expression (a+b)
Parenthesis Does not Match..
```

## 5.5.2 Infix–postfix conversion

An arithmetic expression can be expressed in three ways:

1. The infix expression, which we commonly use:  
A + B is an infix expression where the operator occurs between operands.
2. The prefix expression where the operator precedes the operands:  
+AB is the prefix expression corresponding to the infix expression A+B.
3. The postfix expression where the operator follows the operands:  
AB+ is the corresponding postfix expression.

Postfix expressions are preferred by computers because they are free from operator precedence. However, we use infix expressions when writing a program, and for the ease of evaluations, these are converted to postfix expression by the computer.

A commonly agreed precedence of operators is in Table 5.5.

*Table 5.5 Precedence of operators with priority*

| <i>Operator</i>                               | <i>Priority</i> |
|-----------------------------------------------|-----------------|
| <i>*, unary -, not</i>                        | <i>4</i>        |
| <i>*, /, div, mod, and</i>                    | <i>3</i>        |
| <i>+, -, or</i>                               | <i>2</i>        |
| <i>&lt;, &lt;=, +, &lt; &gt;, &gt;=, &gt;</i> | <i>1</i>        |

The higher the priority, the higher is the precedence. The operators may be different in different languages. Notice that all relational operators have the same precedence.

When we have an expression where the adjacent operators have the same priority, we have to decide which one to evaluate first. If they are exponentiation operators, the evaluation takes place from right to left. For all other equal priority operators, the evaluation takes place from left to right. After the delimiter matching is done, for easy evaluation of an arithmetic expression, the computer converts the infix expression to a postfix expression using a stack.

### Example

Convert the infix expression  $A + B * C$  to a postfix expression (Table 5.6a).

*Table 5.6a Infix to postfix operation*

| <i>Token Read</i> | <i>Stack</i> | <i>Output</i> | <i>Comment</i>           |
|-------------------|--------------|---------------|--------------------------|
| None              | None         | None          | //initialise the stack   |
| A                 |              | A             | // if operand, output it |
| +                 | +            | A             | // if operator, stack it |
| B                 | +            | AB            | // operand: output it    |

The next token is `*`. We have to decide if this is to be pushed onto the stack or the operator `+` has to be popped and written to the output. Since `*` has higher precedence over `+` and has to be evaluated first, we push `*` onto the stack (Table 5.6b).

Table 5.6b Infix to postfix operation

|   |   |     |
|---|---|-----|
| * | + | AB  |
| C | + | ABC |

Since the input expression is exhausted, we output all the remaining elements on the stack and this gives the postfix expression as:

ABC\*+

**Example**

Convert the infix expression  $A/(B+C)*D$  to a postfix expression.

As in the previous example, we initialize a stack and read the tokens of the input infix expression from left to right (Table 5.7a).

Table 5.7a Infix to postfix operation

| Next token | Stack | Output |
|------------|-------|--------|
| None       | None  | None   |
| A          | None  | A      |
| /          | /     | A      |
| (          | /(    | A      |
| B          | /(    | AB     |
| +          | /(+   | AB     |
| C          | /(+   | ABC    |

When we read the next token, which is the right parenthesis ‘)’, we unstack down up to the left parenthesis and delete the left parenthesis from the stack (Table 5.7b).

Table 5.7b Infix to postfix operation

| Next token | Stack | Output  |
|------------|-------|---------|
| )          | /     | ABC+    |
| *          | /*    | ABC+    |
| D          | /*    | ABC+D   |
|            |       | ABC+D*/ |

When the input infix expression is exhausted, we output the contents of the stack and the postfix expression turns out to be

ABC+\*D\*

The following program in Python converts an infix expression to a postfix expression:

### Python program 7

```
def isEmpty(top):
 if (top <= -1):
 return 1
 else:
 return 0

def push(Stack,i,top):
 top += 1
 S.append(i)
 return S,top

def pop(S,top):
 i=S.pop(top)
 return S,top-1,i

def peek(S,top):
 if (isEmpty(top) == 1):
 print("Stack is Empty: Mismatch")
 return 0
 else:
 return S[top]

if __name__ == "__main__":
 Expr=input("Enter an Expression ")
 S=[]
 top = -1
 post = []
 operator = ['*','+','-','/','%']
 result = 0
 for i in range(len(Expr)):
 if ((Expr[i] == '(') or (Expr[i] in operator)):
 S,top = push(S,Expr[i],top)

 elif(Expr[i] == ')'):
 ele= peek(S,top)
 while (ele != '(' and top != -1):
 S,top,item = pop(S,top)
 post.append(item)
 ele= peek(S,top)
 if (ele == '('):
 S,top,item = pop(S,top)
 result = 1
 else:
 result = 0
 break
```

```
elif (Expr[i] == '{' or Expr[i] == '}' or Expr[i] == '['
 or Expr[i] == ']'):
 result = 0
 break

else:
 post.append(Expr[i])

while(top != -1):
 S,top,item = pop(S,top)
 if (item == '('):
 result=0
 break
 post.append(item)
 result = 1

if (result == 1):
 print("The postfix expression is : ",end='')
 for i in post:
 print(i,end=' ')
else:
 print("Parenthesis NOT Matched..")
```

## Output

```
Enter an Expression A/(B+C)*D
The postfix expression is : A B C + D * /
```

### 5.5.3 Evaluation of the postfix expression

After checking for matching delimiters, the infix expression was converted into a postfix expression. The system now evaluates the postfix expression using a stack.

#### Algorithm for evaluation of a postfix expression

1. Initialize a stack
2. Read the next-token from the post-fix expression
3. If it is an operand push it on to the stack
4. If it is an operator pop required number of operands from the stack
5. Apply the operator to the operands and push the result on to the stack
6. If it is “#” indicating end of the stack
7. Pop the result
8. Else go to step 2
9. End.

#### Example

Evaluate the postfix expression 1 3 5 \*+

Table 5.8 Evaluate the postfix expression

| Next token | Stack | Remaining expression    |
|------------|-------|-------------------------|
| -          | -     | 1 3 5 *+##              |
| 1          | 1     | 3 5 *+                  |
| 3          | 1 3   | 5*+                     |
| 5          | 1 3 5 | *+                      |
| *          | 1 15  | + //result = 3*5 = 15   |
| +          | 16    | # // result = 1+15 = 16 |

See Table 5.8. Now we have reached the end of expression, so pop the contents of the stack. And the value of the postfix expression is 16.

A simple program in Python is given next to achieve this.

### Python program 8

```
def isEmpty(top):
 if (top <= -1):
 return 1
 else:
 return 0

def push(S,i,top):
 top += 1
 S.append(i)
 return S,top

def pop(S,top):
 i = S.pop(top)
 return S,top-1,i
```

The functions `isEmpty()`, `push()` and `pop()` are as defined earlier.

### Python program 9

```
def calculate(op1,op2,i):
 if i is '*':
 return float(op1)*float(op2)
 elif i is '/':
 return float(op1)/float(op2)
 elif i is '+':
 return float(op1)+float(op2)
 elif i is '-':
 return float(op1)-float(op2)
 else:
 return float(op1)%float(op2)
```

The `calculate(op1,op2,i)` function takes three parameters. The first two are two operands and the third one is an operator. Depending on the character received by the operator, the operation is performed.

**Program continues...**

```
if __name__ == "__main__":
 Expr=input("Enter an Expression ")
 S=[]
 top = -1
 operator = ['*','+','-','/','%']
 for i in range(len(Expr)):
 result = 0
 if (Expr[i] in operator):
 S,top,item = pop(S,top)
 S,top,item1 = pop(S,top)
 result= str(calculate(item1,item,Expr[i]))
 S,top=push(S,result,top)
 elif(Expr[i] in '0123456789'):
 S,top=push(S,Expr[i],top)
 else:
 pass

 S,top,item = pop(S,top)
 print("The result of postfix expression ",Expr, "
 is ", item)
```

The main program accepts a postfix expression in the variable `Expr`. Initialize an empty list `S` that is used as a stack. And a variable `top` is initialized to -1. All the arithmetic operators are listed in a list with variable name `operator`.

Read each character of the expression from left to right until the end of the expression. If an numeric value appeared, we pushed it onto the stack. If an operator appeared, we pop top two elements from the stack, perform the calculation and again push the result onto the stack. When the last element of the expression is reached, pop the element from the stack and show the result.

## Output

```
Enter an Expression 532+/8*
The result of postfix expression 532+/8* is 8.0
```

### 5.5.4 Recursion

Let us first understand what happens when a function is called. If the function has formal parameters, they are initialized with the actual parameters

passed to the function. In addition, the system has to know where to resume the execution of the program after the completion of the function call. In other words, the system has to remember where the function has been called from. This could be done by storing the return address in main memory, set aside to store return addresses. However, we do not know, in advance, how much space might be needed. Moreover, more information might be needed to be stored, besides the return address. Therefore, dynamic allocation using a run-time stack might be a better solution. For example, the local variables need to be stored. If a function *f* has a local variable *x* and it calls a function *g* that has a local variable also named *x*, the system must be able to distinguish them. Thus the state of each function including *main()* is characterized by the parameters passed to the function, its local variables and the return address from where we start further execution of the calling function. The data record that contains all this information is called the activation record and is stored on a run-time stack. This activation record exists as long as the corresponding function is executing. The activation record of a called function disappears before the activation record of the calling function disappears. Thus, the activation record of the *main()* function outlives all other activation records.

The activation record of any function contains the following information:

1. Values of the parameters of the function
2. Information about local variables
3. The return address
4. A pointer, called SP (stack pointer), to the callers activation record
5. The returned value of the function that is not declared as void

The activation record of the executing function is at the top of the stack. For example, if the main function calls *f1()*, *f1()* calls *f2()* and *f2()* calls *f3()*, the run-time stack, when *f3()* is being executed is as in Figure 5.11.

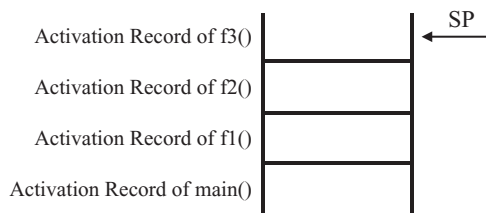


Figure 5.11 Activation record

Now, if we call a function that has the same name as the calling function, we call it recursion. To put it more precisely, it is an instantiation of a function calling another instantiation of the same function. These invocations

of the function are represented by different activation records. Let us take a simple example using a program written in Python:

**Python program 10**

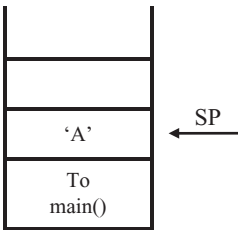
```
def rev():
 i=0
 ch=input()
 if (ch != 'z'):
 rev()
 print(ch)

if __name__ == "__main__":
 rev()
```

**Output**

A  
b  
c  
z  
z  
c  
b  
a

When the `rev()` function is called an activation record is created with the variable 'ch' and the return address. There is no need to store the result since no value is returned. And this is indicated by prefixing the function with void. The function `rev()` reads the first character and the contents of the run-time stack just before `rev()` calls itself for the first time (Figure 5.12a).



*Figure 5.12a* Contents of the run-time stack

The second character is read and after checking that it is not 'z', `rev()` is called again which reads character 'b'. Now the activation record for the second call is pushed onto the run-time stack (Figure 5.12b).

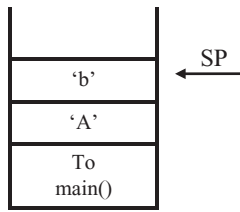


Figure 5.12b Contents of the run-time

In a similar manner, all the elements are pushed onto the stack and finally when 'z' is read the run-time stack is as shown in Figure 5.12c.

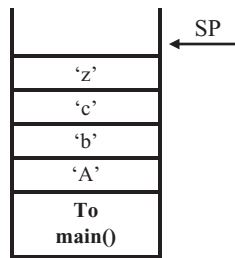


Figure 5.12c Contents of the run-time stack

When the system finds the end of line character, no other statement is executed. It then retrieves the return address from the activation record and discards this record by decrementing the stack pointer by the proper number of bytes.

Since the activation record of the fourth call to `rev()` is active, the value of `ch`, which is 'z', is output as the first character and the activation record for the fourth call is discarded by decrementing the stack pointer. 'c' is assigned to `ch` and then printed. Finally the activation record of the first call is reached and A is printed and what is seen on the screen is "zcbA".

Let us consider a non-recursive version of reversing a string:

### Python program 11

```
if __name__ == "__main__":
 Expr=input("Enter an Expression ")
 S=[]
 top = -1
 i=0
 while(i < len(Expr)):
 S,top = push(S,Expr[i],top)
 i+=1
```

```
i=0
print("The Reverse of the String is :",end='')
while(i < len(Expr)):
 S,top,item = pop(S,top)
 print(item,end='')
 i+=1
```

### Output

Enter an Expression Rakesh  
The Reverse of the String is :hsekaR

### True-false questions

- Stack is also known as an LIFO data structure.
  - True
  - False
- A printer uses a stack data structure.
  - True
  - False
- While pushing an element on to a stack, we need to increment the top by 1 and then insert the element at the top location.
  - True
  - False
- Peek operation does not delete the top.
  - True
  - False
- A stacked can be used to convert a decimal number to a binary number.
  - True
  - False

### Fill-in-the-blank questions

- The \_\_\_\_\_ function first checks if the stack is empty.
- The \_\_\_\_\_ function first checks if the stack is full.
- The isEmpty() function checks if the top is at \_\_\_\_\_.
- In stack insertion/deletion is done at \_\_\_\_\_ end.
- If Create(S) creates an empty stack, then pop(create(S)) returns \_\_\_\_\_.

### Multiple-choice questions

- The best data structure to check whether a arithmetic expression is well parenthesized or not is a
  - Queue
  - Stack
  - Tree
  - List
- A single array A[1.. Max] is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables top1 and top2 (top1 > top2 ) point to the location of the topmost element in each

stack. If the space is to be efficiently used, the condition for stack full is

- a.  $(\text{top1} = \text{Max}/2) \text{ and } (\text{top} = \text{max}/2+1)$
  - b.  $\text{top1} + \text{top2} = \text{Max}$
  - c.  $(\text{top1} = \text{Max}/2) \text{ and } (\text{top} = \text{max})$
  - d.  $\text{top1} = \text{top2} - 1$
3. A linear set of elements in which deletion can be done from one end (*front*) and insertion can take place only at the other end is known as a
- a. Queue
  - b. Stack
  - c. Tree
  - d. Branch
4. The end at which a new element gets added to a queue is called the
- a. Front
  - b. Rear
  - c. Top
  - d. Bottom
5. The advantage of recursion is that the
- a. Code size is less.
  - b. Time complexity is less.
  - c. Space complexity is less.
  - d. None of the above.

6. Consider the following code:

```
def f(n):
 i=1
 if (n >= 5):
 return n
 else:
 n = n + i
 i += 1
 return (f(n))
```

What would be the value returned by  $f(1)$ ?

- a. 5
  - b. 6
  - c. 7
  - d. 8
7. Any recursive function can be converted into an equivalent non-recursive function
- a. Always
  - b. Never
  - c. Sometimes
  - d. If the function is tail recursive

8. The Fibonacci function  $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$  is an example of
- Direct recursion
  - Tree recursion
  - Linear recursion
  - Both a and b
9. With what data structure can reversing of string can be easily implemented?
- Array
  - List
  - Stack
  - All of the above
10. Find the output of the sample code.
- ```
def fun(n)
    Stack S // Say it creates an empty stack S
    while(n > 0):
        // This line pushes the value of n%2 to stack S
        push(S, n%2)
        n = int(n/2)
    // Run while Stack S is not empty
    while(!isEmpty(S)):
        print(pop(S)); // pop an element from S and print it
```
- Prints the value of $\log n$
 - Prints the binary representation of n
 - Prints the reverse of the number
 - All of the above
11. The following postfix expression with single-digit operands is evaluated using a stack: $8\ 2\ 3\ ^\wedge\ /\ 2\ 3\ *\ +\ 5\ 1\ *\ -$
- Note that $^\wedge$ is the exponentiation operator. The top two elements of the stack after the first $*$ is evaluated are:
- 1,5
 - 3,2
 - 5,7
 - 6,1
12. If the elements A, B, C and D are placed in a stack and are deleted one at a time, in what order will they be removed?
- ABCD
 - DCBA
 - DCAB
 - ADBC
13. In linked list implementation of a queue, where can a new element be inserted?
- At the head of link list
 - At the tail of the link list
 - At the center position in the link list
 - None of the above

14. The following operations are performed on a stack: push(A), push(B), push(C), pop(), pop(), push(D), pop(), pop(). What is on the top of the stack?

- | | |
|------|---------|
| a. A | c. D |
| b. C | d. NULL |

15. When several jobs arrive at the printer, the jobs join a

- | | |
|------------|----------------------|
| a. Queue | c. Stack |
| b. Dequeue | d. None of the above |

16. A single array A[1..MAXSIZE] is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables top1 and top2 (top1 < top2) point to the location of the topmost element in each of the stacks. If the space is to be used efficiently, the condition for "stack full" is

- | | |
|--|--|
| a. $\text{top1} + \text{top2} = \text{MAXSIZE}$ | c. $\text{top1} = \text{top2} - 1$ |
| b. $\text{C} (\text{top1} = \text{MAXSIZE}/2)$
and $(\text{top2} = \text{MAXSIZE}/2 + 1)$ | d. $(\text{top1} = \text{MAXSIZE}/2) \text{ or } (\text{top2} = \text{MAXSIZE})$ |

17. Assume that the operators +, -, $\times$ are left associative and $\wedge$ is right associative. The order of precedence (from highest to lowest) is $\wedge$, $\times$, +, -. The postfix expression corresponding to the infix expression $a + b \times c - d \wedge e \wedge f$ is

- | | |
|--|--|
| a. $- + a \times bc \wedge \wedge def$ | c. $abc \times + de \wedge f \wedge -$ |
| b. $ab + c \times d - e \wedge f \wedge$ | d. $abc \times + def \wedge \wedge -$ |

18. The priorities of arithmetic operators +, *, $\wedge$, / can be

- | | |
|---------------|---------------|
| a. 5, 1, 2, 9 | c. 1, 2, 5, 2 |
| b. 9, 5, 2, 1 | d. 5, 3, 3, 4 |

19. A normal stack of size MAX gets full when

- | | |
|----------------------------------|--|
| a. $\text{Top} = \text{MAX} - 1$ | c. $\text{Top} = -1$ |
| b. $\text{Top} = \text{NULL}$ | d. $\text{Top} = (\text{No of element} + 1) \% \text{MAX}$ |

20. A stack can be implemented using a

- | | |
|---------------|---------------------|
| a. Dictionary | c. List |
| b. Class | d. All of the above |

Descriptive questions

- 1. Construct a stack pushing the elements 3 5 7 6 8 in that order. Find the status of the stack after the following operations:
Push(9), pop(),pop(),pop(),push(10)
- 2. Convert the decimal number 24 to its binary equivalent using stack.
- 3. Convert $Z - ((X * ((X + Y/J - 2)) + Y)/3)$ into a postfix expression.
- 4. Reverse the string ABCDFEGH using a stack.
- 5. Evaluate the following postfix expressions.
 - a. 70 14 4 5 15 3 /* - - /6+
 - b. 3 5 6 *+13-18 2/+
- 6. Write the equivalent infix expression for the following postfix expressions.
 - a. AB+CD-*
 - b. AB-C-D*
- 7. Transform the following infix expression into the equivalent infix expression: +A-BC

Answers to true–false questions

- 1. True
- 2. False
- 3. True
- 4. True
- 5. True

Answers to fill-in-the-blank questions

- 1. pop()
- 2. push()
- 3. -1
- 4. Same
- 5. False

Answers to multiple-choice questions

- | | | | | | | | | | |
|-----|---|-----|---|-----|---|-----|---|-----|---|
| 1. | b | 2. | d | 3. | a | 4. | b | 5. | a |
| 6. | a | 7. | a | 8. | d | 9. | c | 10. | b |
| 11. | d | 12. | b | 13. | b | 14. | d | 15. | a |
| 16. | c | 17. | d | 18. | c | 19. | a | 20. | d |